

ALICE Track B · GitHub setup

Create your team repository and push it to GitHub.

Two things to check first.

1) On **Windows**, use **Git Bash** (it installs with Git), not CMD or PowerShell, so every command below works exactly as written. On Mac or Linux, use the **Terminal**. 2) Everywhere you see **alice-track-b-team-XX**, replace **XX** with your team number, and replace **YOUR_USERNAME** with your GitHub username.

0. One-time setup on your computer

Do this once, ever. Skip if you already have Git and a GitHub account.

Check that Git is installed

```
git --version
```

If you see a version number, you are good. If you see command not found, install Git from git-scm.com/downloads, then reopen the terminal (Windows users: open **Git Bash**).

Tell Git who you are

This is required before your first commit. If you skip it, Git stops with *"Please tell me who you are"*.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Have a GitHub account

Create a free account at github.com if you do not have one. Remember your **username**, you will need it below.

1. Create the empty repository on GitHub (in your browser)

1. Go to github.com, click the **+** at the top right, then **New repository**.
2. Repository name: **alice-track-b-team-XX** (use your team number).
3. **Important:** leave it empty. Do not tick "Add a README", "Add .gitignore", or "Choose a license". You will create the README yourself in the next step. Adding one here causes a push error later.
4. Click **Create repository**. Keep this page open, you will need the URL.

2. Create the project on your computer

Open Git Bash (Windows) or Terminal (Mac/Linux) and paste this whole block. It makes your project's folders, a README, and the first commit — using the same structure we've used all session (backend + frontend, each with their own sub-folders).

```
mkdir alice-track-b-team-XX
cd alice-track-b-team-XX

mkdir notebooks backend backend/services frontend frontend/pages frontend/utils
mkdir models data/raw data/sample_test
touch notebooks/.gitkeep backend/.gitkeep backend/services/.gitkeep
touch frontend/.gitkeep frontend/pages/.gitkeep frontend/utils/.gitkeep
touch models/.gitkeep data/.gitkeep data/raw/.gitkeep data/sample_test/.gitkeep
echo "# ALICE Track B - Team XX" > README.md

git init
git add .
git commit -m "Initial project structure"
git branch -M main
```

What did this do?

It created one folder per part of the project: notebooks (the Colab work), backend and its services/ sub-folder (the FastAPI server and its prediction/explainability/prioritization logic), frontend with its pages/ and utils/ sub-folders (the Streamlit dashboard), models (the exported .pkl/.json files), and data with raw/ and sample_test/ sub-folders. It added an empty .gitkeep in each so Git keeps the empty folders, wrote a short README, and saved your first snapshot (the "commit").

3. Connect to GitHub and push

Replace **YOUR_USERNAME** and **XX**, then paste.

```
git remote add origin https://github.com/YOUR_USERNAME/alice-track-b-team-XX.git
git push -u origin main
```

4. When it asks for a username and password

Your GitHub password will NOT work here.

GitHub removed password login for Git. If you type your account password you will get *"Authentication failed"*. Use a **Personal Access Token** as the password instead.

Create the token once:

1. On github.com: click your avatar, top right, then **Settings**.
2. Scroll down to **Developer settings** (very bottom of the left menu).
3. **Personal access tokens** then **Tokens (classic)** then **Generate new token (classic)**.
4. Give it a name, set an expiration, and tick the **repo** checkbox (that scope is enough).
5. Click **Generate token** and **copy it now** (you will not see it again).

Back in the terminal, when Git prompts:

Username: your GitHub username. **Password:** paste the token. Nothing appears on screen while you paste or type, that is normal. Press Enter.

Faster option if the command gh is installed.

Run `gh auth login` once, choose GitHub.com then HTTPS, and authenticate in the browser. After that, `git push` just works with no token prompts.

5. Add your teammates to the repository

So the whole team can push to the same repo:

1. On your repository page: **Settings** then **Collaborators** (you may be asked for your password or token).
2. **Add people**, type each teammate's GitHub username, and invite them. They accept from their email or notifications.
3. Each teammate then copies the repo to their machine with:

```
git clone https://github.com/YOUR_USERNAME/alice-track-b-team-XX.git
cd alice-track-b-team-XX
```

6. Everyday workflow (use this every time)

After you change any file, save your work to GitHub with three commands.

```
git add .
git commit -m "describe what you changed"
git push
```

Before you start working, get your teammates' latest changes first:

```
git pull
```

7. If something goes wrong

Message you see	What it means / how to fix
Please tell me who you are	You skipped step 0. Run the two git config --global commands, then redo the commit.
Authentication failed / Support for password authentication was removed	You used your account password. Use a Personal Access Token as the password (step 4).
remote origin already exists	You ran the remote add twice. Fix the URL instead: <code>git remote set-url origin https://github.com/YOUR_USERNAME/alice-track-b-team-XX.git</code>
repository not found	The username or repo name is wrong, or you did not create the repo on GitHub (step 1), or your token is missing the repo scope. Check the exact spelling.
Updates were rejected / fetch first / non-fast-forward	The GitHub repo was not empty (you added a README or license there in step 1). Run once: <code>git pull origin main --allow-unrelated-histories</code> then git push. Or delete the repo on GitHub and recreate it empty.
src refspec main does not match any	You have no commit yet. Run the commit from step 2, then git branch -M main, then push.
touch: command not found (Windows)	You are in CMD or PowerShell. Open Git Bash and paste the commands there.
It opens a strange text editor after a merge	That is the commit-message editor (Vim). Type :wq and press Enter to save and close.

8. Whole thing at a glance

For reference once you understand each step. Replace **YOUR_USERNAME** and **XX**, and make sure you created the empty repo on GitHub first.

```
# one time on your machine
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# build the project
mkdir alice-track-b-team-XX
cd alice-track-b-team-XX
mkdir notebooks backend backend/services frontend frontend/pages frontend/utils
mkdir models data data/raw data/sample_test
touch notebooks/.gitkeep backend/.gitkeep backend/services/.gitkeep
touch frontend/.gitkeep frontend/pages/.gitkeep frontend/utils/.gitkeep
touch models/.gitkeep data/.gitkeep data/raw/.gitkeep data/sample_test/.gitkeep
echo "# ALICE Track B - Team XX" > README.md
git init
git add .
git commit -m "Initial project structure"
git branch -M main

# connect and push (create the empty repo on GitHub first)
git remote add origin https://github.com/YOUR_USERNAME/alice-track-b-team-XX.git
git push -u origin main

# every day after that
git add .
git commit -m "describe what you changed"
git push
```